

Calculadora edad / Actividad 1

Alexis Maier

Aramis Demaria

Leon Melgarejo

EESTN1

Curso: Laboratorio de Programación 5°

Profesor: Mansilla Muñoz York E.

Fecha: 07/10/2026

Introducción

En el marco de las Actividades de la materia Laboratorio de Programación, se aborda una revisión integral de conceptos lógicos y algorítmicos mediante la implementación de una calculadora de edad exacta. El propósito principal radica en analizar las diferencias operativas, sintácticas y de nivel entre un lenguaje interpretado de alto nivel como Python y un lenguaje estructurado de bajo nivel como C++, aplicando filtros de consistencia y estructuras de control temporal del calendario real.

Desarrollo

En computadoras locales con permisos de administrador, se utiliza Visual Studio Code en conjunto con el compilador MSYS2 para dar soporte a C++. En entornos restringidos de laboratorio o equipos escolares bloqueados, se recurre a la plataforma web integrada OnlineGDB, permitiendo la compilación en la nube y la captura de resultados para la carpeta de campo.

Estructura del Proyecto

El proyecto se organiza bajo un esquema de control de versiones preparado para GitHub, estructurado de la siguiente manera:

`calculadoraedad`: Raíz del repositorio.

`.gitignore`: Filtro de exclusión para archivos pesados de compilación.

`README.md`: Documentación de lectura rápida del ejercicio.

`docs`/El presente informe técnico.

`proyecto_cpp`: Código fuente del esqueleto en C++.

proyecto_python: Código fuente del esqueleto en Python.

Variables y Tipos de Datos

Ambos lenguajes procesan la información del calendario mediante variables de tipo entero (`int`), representando el día, el mes y el año tanto para los datos actuales del sistema (`diaA`, `mesA`, `anioA`) como para los ingresados por el usuario (`diaN`, `mesN`, `anioN`). Mientras que C++ requiere una declaración explícita de tipos estáticos al inicio del contexto, Python gestiona el tipado de manera dinámica y requiere conversiones explícitas (`int(input())`) al recibir cadenas de texto de la consola.

Estructuras de Control y Filtros de Consistencia

La lógica interna implementa un filtro de consistencia: una serie de condiciones booleanas encargadas de validar que la fecha ingresada exista en el calendario real. Se descartan años fuera del rango (1900-2026), meses inválidos (`< 1` o `> 12`) y días fuera de los límites mensuales estándar (30 o 31 según corresponda).

La lógica del año bisiesto calcula si un año posee 366 días para alterar el límite de febrero a 29 días. Su fórmula fundamental utiliza operadores aritméticos de módulo (%) y relacionales lógicos (&&, ||, !):

codigo python

```
# Materia: Laboratorio de Programacion (LPR)

# E.E.S.T. N° 1 "Eduardo Ader" - Vicente Lopez

# Archivo: proyecto_python/Act 1 - calculadoraEdad.py

# Actividad 1: De Revision Integral (Python & C++)

def es_bisiesto(anio):

    return (anio % 4 == 0 and anio % 100 != 0) or (anio % 400 == 0)

def es_fecha_valida(d, m, a):

    if a < 1900 or a > 2026: return False

    if m < 1 or m > 12: return False
```

```

    if d < 1 or d > 31: return False

    # CORREGIDO: Se agregó la lista [4, 6, 9, 11] que faltaba entre
    'in' y 'and'

    if m in [4, 6, 9, 11] and d > 30: return False

    if m == 2:

        if es_bisiesto(a):

            if d > 29: return False

        else:

            if d > 28: return False

    return True

# Codigo principal

print("=====")

print("  CALCULADORA EN PYTHON - INTEGRANTES:")

print("    1) Alexis Maier")

print("    2) Aramis Demaria")

print("    3) Leon Melgarejo")

print("=====")

try:

    diaN = int(input("Ingrese Dia de nacimiento: "))

    mesN = int(input("Ingrese Mes de nacimiento: "))

    anioN = int(input("Ingrese Anio de nacimiento: "))

```

```

diaA, mesA, anioA = 8, 7, 2026

if not es_fecha_valida(diaN, mesN, anioN):

    print("[ERROR] La fecha ingresada no es valida.")

else:

    edad = anioA - anioN

    # COMPLETADO: Si el mes de nacimiento es posterior al mes
actual, o si

    # estamos en el mismo mes pero todavia no llego el dia de
cumpleanos,

    # la persona aun no cumplio anos este anio, por eso se le resta
1.

    if mesN > mesA or (mesN == mesA and diaN > diaA):

        edad = edad - 1

    print(f"\n[SISTEMA] Fecha de hoy: {diaA}/{mesA}/{anioA}")

    print(f"[SISTEMA] Edad calculada: {edad} anos.")

except ValueError:

    print("[ERROR] Deben ingresar numeros enteros.")

```

Ajuste de Edad Exacta (Resolución del Desafío)

Para determinar la edad exacta restando los años (`anioA - anioN`), se debe programar un condicional que reste un año si el individuo aún no ha cumplido años en el ciclo actual.

codigo c++

```
/*
=====
Materia: Laboratorio de Programacion (LPR)
E.E.S.T. N° 1 "Eduardo Ader" - Vicente Lopez
Curso: 5° Año 3° Division | Profesor: Prof. York
Archivo: proyecto_cpp/Act 1 - calculadoraEdad.cpp
Actividad 1: De Revision Integral (Python & C++)
Objetivo: Calcular la edad exacta validando el calendario real.
=====
*/
#include <iostream>
using namespace std;

// 1. Evalua si un año es bisiesto
bool esBisiesto(int anio) {
    return (anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0);
}

// 2. Filtro de consistencia de calendario
bool esFechaValida(int d, int m, int a) {
    if (a < 1900 || a > 2026) return false;
    if (m < 1 || m > 12) return false;
    if (d < 1 || d > 31) return false;
    // Meses de 30 días
    if ((m == 4 || m == 6 || m == 9 || m == 11) && d > 30) return
false;
    // Caso especial de Febrero
    if (m == 2) {
        if (esBisiesto(a)) {
            if (d > 29) return false;
        } else {
            if (d > 28) return false;
        }
    }
}
```

```

    }

    return true;
}

int main() {
    int diaN, mesN, anioN;

    int diaA = 8, mesA = 7, anioA = 2026; // Fecha actual para el
ejercicio

    //
=====

    // REGLA OBLIGATORIA (EVITA EL PLAGIO):
    // Reemplacen "Estudiante" por su nombre y apellido reales en la
salida.
    //
=====

    cout << "===== " <<
endl;

    cout << "  CALCULADORA DE EDAD - INTEGRANTES:" << endl;
    cout << "    1) Alexis Maier" << endl;
    cout << "    2) Aramis Demaria" << endl;
    cout << "    3) Leon Melgarejo" << endl;
    cout << "===== " <<
endl;

    cout << "Ingrese Dia, Mes y Anio de nacimiento (separados por
espacios): ";
    cin >> diaN >> mesN >> anioN;

    if (!esFechaValida(diaN, mesN, anioN)) {
        cout << "[ERROR] La fecha ingresada no existe en el
calendario." << endl;
        return 1;
    }

    int edad = anioA - anioN;

    // COMPLETADO: Si el mes de nacimiento es posterior al mes actual,

```

```

o si
    // estamos en el mismo mes pero todavia no llego el dia de
cumpleanos,
    // la persona aun no cumplio anos este anio, por eso se le resta 1.
    if (mesN > mesA || (mesN == mesA && diaN > diaA)) {
        edad = edad - 1;
    }

    cout << "\n[SISTEMA] Fecha de hoy: " << diaA << "/" << mesA << "/"
<< anioA << endl;
    cout << "[SISTEMA] Edad calculada: " << edad << " anos." << endl;
    cout << "===== " <<
endl;
    return 0;
}

```

Conclusión

La realización de esta actividad de revisión permitió consolidar los fundamentos lógicos aplicados a dos tecnologías con filosofías opuestas. El uso de estructuras modulares basadas en *scaffolding* (esqueletos) facilitó el enfoque directo sobre las reglas del negocio y el control de errores (filtros de consistencia) sin necesidad de reescribir flujos de entrada y salida estándar. El desarrollo bajo metodologías híbridas (locales u online) y la integración obligatoria de portafolios digitales en GitHub promueven habilidades esenciales de organización y transparencia en el desarrollo de software profesional.

<https://github.com/letan-doto07/calculadoraedad/tree/main>